



# F70L ELTASHFEER

## Implementation Report

|                         |                    |
|-------------------------|--------------------|
| <b>Course:</b>          | Cryptography       |
| <b>Instructor:</b>      | Eng. Eman Yasser   |
| <b>Submission Date:</b> | 16 May 2026        |
| <b>Team Members:</b>    | Team of 5 Students |

|                            |         |
|----------------------------|---------|
| Mahmoud Mostafa Foad       | 2405005 |
| Omar Nasrallah Saad        | 2405037 |
| Omar Ramy                  | 2405264 |
| Mamdouh Mohamed<br>Mamdouh | 2405030 |
| Mohamed Ibrahim Aboyoussef | 2405296 |

# Section 1 – Algorithms Used

---

This section provides a concise explanation of every cryptographic algorithm and encoding technique implemented in the Crypto Toolkit application.

## 1.1 Symmetric Encryption

Symmetric encryption uses the same secret key for both encryption and decryption. The sender and receiver must share this key securely in advance.

### AES – Advanced Encryption Standard

AES is the current gold-standard symmetric cipher, adopted by NIST in 2001. It operates on 128-bit blocks and supports key sizes of 128, 192, or 256 bits. The algorithm performs multiple rounds of substitution, permutation, and mixing on the data, making it extremely resistant to brute-force attacks.

- ✓ Block size: 128 bits | Key sizes: 128 / 192 / 256 bits
- ✓ Number of rounds: 10 (128-bit key) up to 14 (256-bit key)
- ✓ Status: Widely used, considered secure for the foreseeable future.

### DES – Data Encryption Standard

DES was the first widely adopted symmetric cipher, standardised in 1977. It uses a 56-bit effective key on 64-bit data blocks. Due to its short key length it is now considered insecure – it can be broken by brute force in hours with modern hardware.

- ⚠ Block size: 64 bits | Effective key size: 56 bits
- ⚠ Rounds: 16 Feistel rounds
- ⚠ Status: Deprecated – included here for educational comparison only.

### 3DES – Triple DES

3DES applies the DES algorithm three times to each block, significantly increasing security over plain DES. The EDE (Encrypt-Decrypt-Encrypt) mode uses up to three independent keys, giving an effective key length of 112 or 168 bits.

- ✓ Block size: 64 bits | Key size: 112 or 168 bits (EDE mode)
- ✓ Rounds: 48 total (3 × 16 Feistel rounds)
- ✓ Status: Being phased out in favour of AES, but still acceptable in legacy systems.

| Algorithm | Key Size        | Block Size | Rounds | Security Status |
|-----------|-----------------|------------|--------|-----------------|
| AES       | 128/192/256 bit | 128 bit    | 10–14  | ✓ Secure        |
| DES       | 56 bit          | 64 bit     | 16     | ✗ Broken        |
| 3DES      | 112/168 bit     | 64 bit     | 48     | ⚠ Legacy        |

## 1.2 Asymmetric Encryption – RSA

RSA (Rivest–Shamir–Adleman, 1977) is the most widely used public-key cryptosystem. Unlike symmetric encryption, RSA uses a mathematically linked key pair: a Public Key (shared openly) and a Private Key (kept secret).

### How RSA Works

- 1. Key Generation: Choose two large primes  $p$  and  $q$ . Compute  $n = p \times q$ . Choose  $e$  (public exponent, typically 65537). Compute  $d$  (private exponent) as the modular inverse of  $e$ .
- 2. Encryption: Ciphertext  $C = M^e \bmod n$  (using the Public Key)
- 3. Decryption: Plaintext  $M = C^d \bmod n$  (using the Private Key)

*The security relies on the difficulty of factoring the large number  $n = p \times q$ .*

In our implementation we use JSEncrypt with a 2048-bit key size. The keys are returned in PEM (Privacy-Enhanced Mail) format – a Base64-encoded representation of the DER binary structure, wrapped in BEGIN/END headers.

## 1.3 Encoding & Decoding

Encoding transforms data into a different representation for safe transmission or storage. It is NOT encryption – no key is required and anyone can reverse it.

### Base64

Base64 maps every 3 bytes of binary data to 4 printable ASCII characters drawn from a 64-character alphabet (A–Z, a–z, 0–9, +, /). It is used in email attachments, data URIs, and JSON payloads where binary data must travel through text-only channels.

```
Encode: btoa(unescape(encodeURIComponent(text)))
Decode: decodeURIComponent(escape(atob(encoded)))
```

### Hex (Hexadecimal)

Hex encoding represents each byte of data as a two-digit hexadecimal number (00–FF). It is commonly used in debugging, network packet dumps, and cryptographic key display.

```
Encode: each char → charCode → hex pair (e.g. 'A' → 65 → '41')
Decode: each hex pair → parseInt(hex,16) → String.fromCharCode(n)
```

## URL Encoding (Percent Encoding)

URL encoding replaces characters that are not safe in a URL with a percent sign followed by the two-digit hex code of the character's UTF-8 byte (e.g. space → %20, & → %26).

```
Encode: encodeURIComponent(text)
Decode: decodeURIComponent(encoded)
```

## 1.4 Hashing

A hash function is a one-way mathematical function that maps arbitrary-length input to a fixed-length digest. It is deterministic (same input → same output) but irreversible (you cannot recover the input from the hash).

### SHA-256

SHA-256 is part of the SHA-2 family designed by the NSA. It produces a 256-bit (64 hex character) digest. It is used in SSL/TLS certificates, Bitcoin, and password storage.

- ✓ Output size: 256 bits (64 hex characters)
- ✓ Collision resistance: no practical attack known

### SHA-512

SHA-512 also belongs to the SHA-2 family, producing a 512-bit (128 hex character) digest. It offers a higher security margin and is faster on 64-bit processors than SHA-256.

- ✓ Output size: 512 bits (128 hex characters)
- ✓ Preferred for high-security or long-term archival use.

## Salted Hashing

Salted hashing adds a unique random value (the salt) to the input before hashing. Even if two users have the same password, their stored hashes will be different.

- ✓ Prevents Rainbow Table attacks – pre-computed hash lookup tables.
- ✓ Salt is generated using `window.crypto.getRandomValues()` – a cryptographically secure random source.
- ✓ Storage format: `SALT || SHA-256(salt + password)`

```
salt = crypto.getRandomValues(16 bytes) → hex string
hash = SHA256(salt + input)
store = { salt, hash }
```

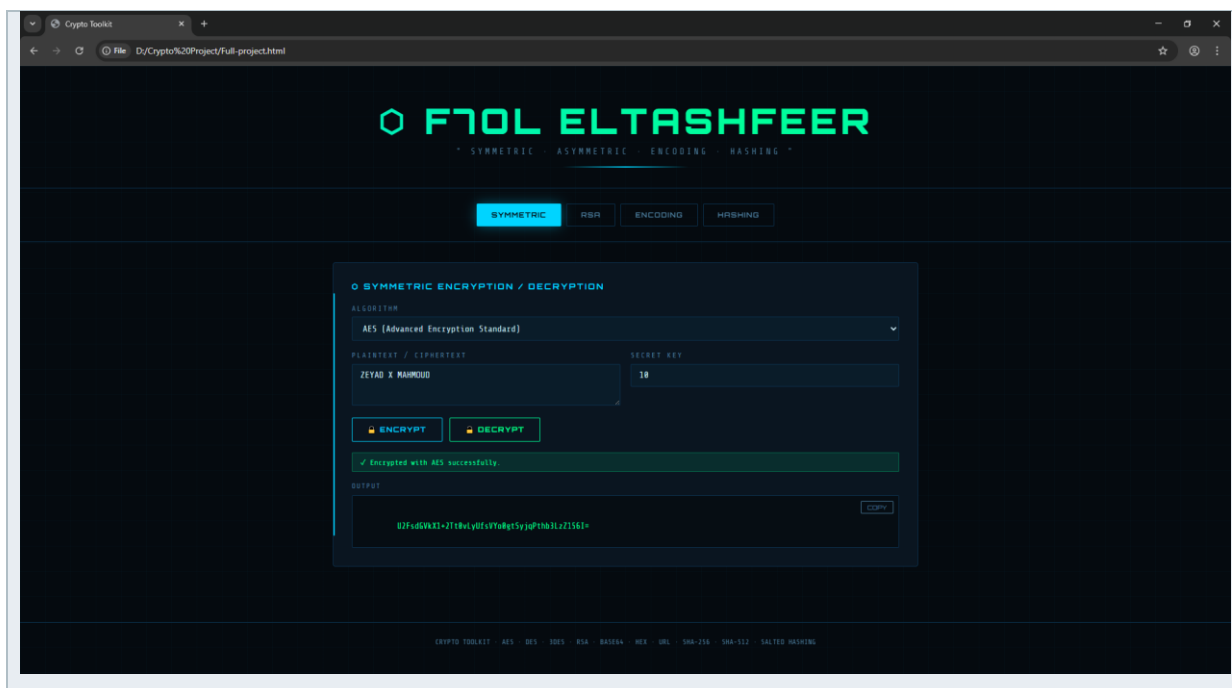
*To verify a password later: re-hash (stored\_salt + entered\_password) and compare with the stored hash.*

## Section 2 – Implementation Screenshots

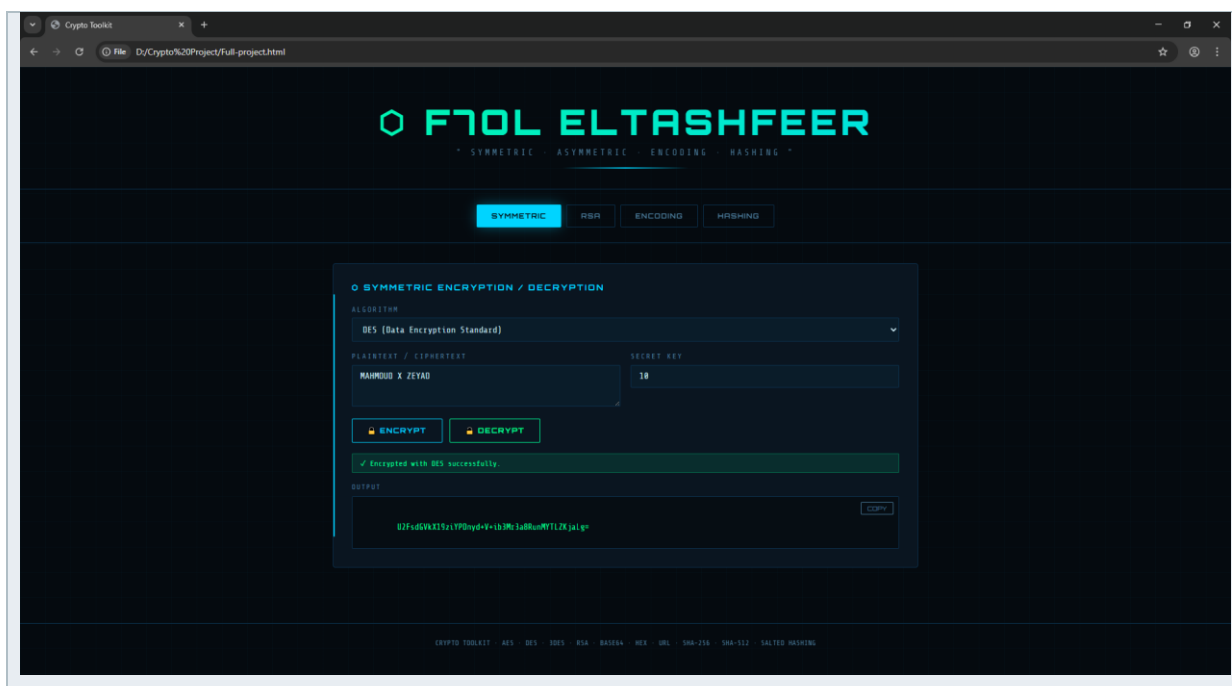
The following placeholders mark where screenshots of the live application should be inserted. Refer to the instructions at the end of this document for guidance on taking and inserting screenshots.

### 2.1 Encryption Screenshots

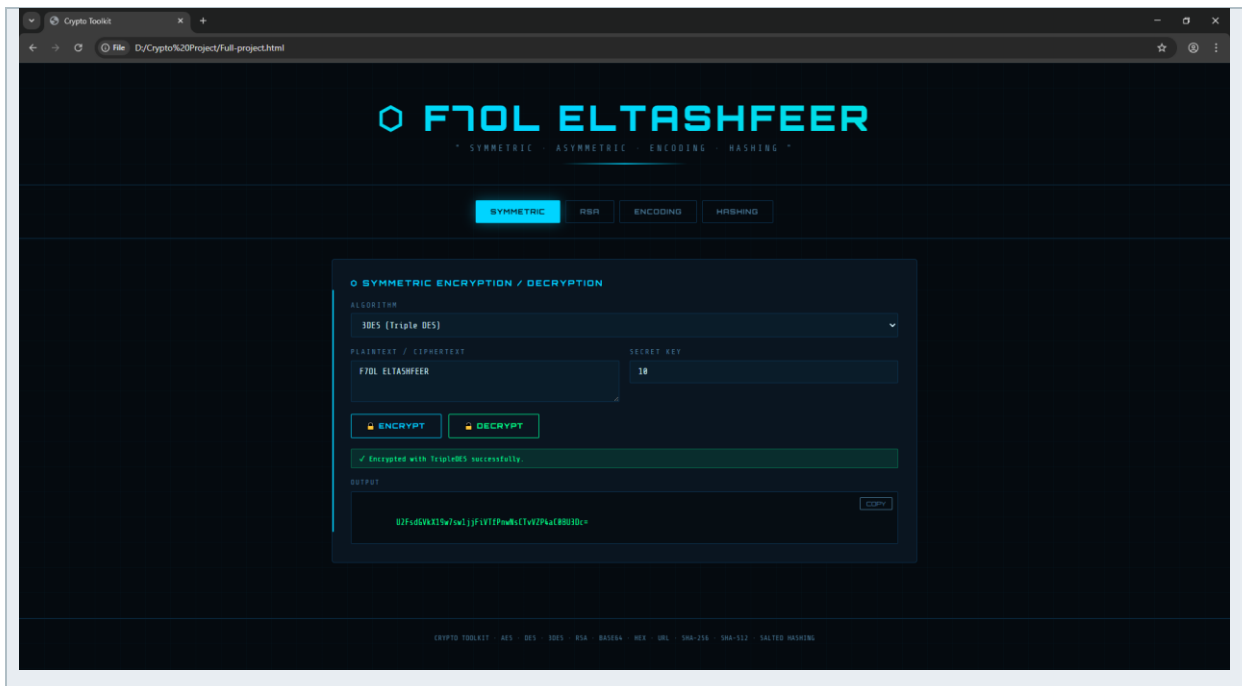
#### AES Encryption



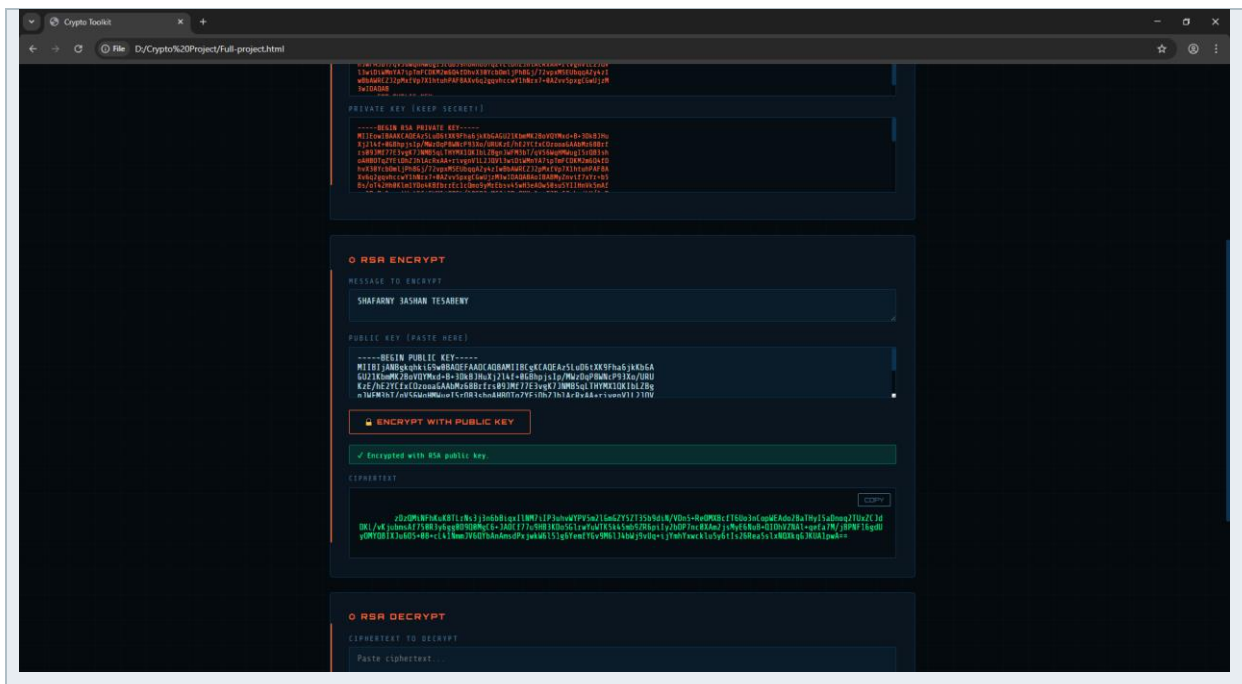
#### DES Encryption



## 3DES Encryption

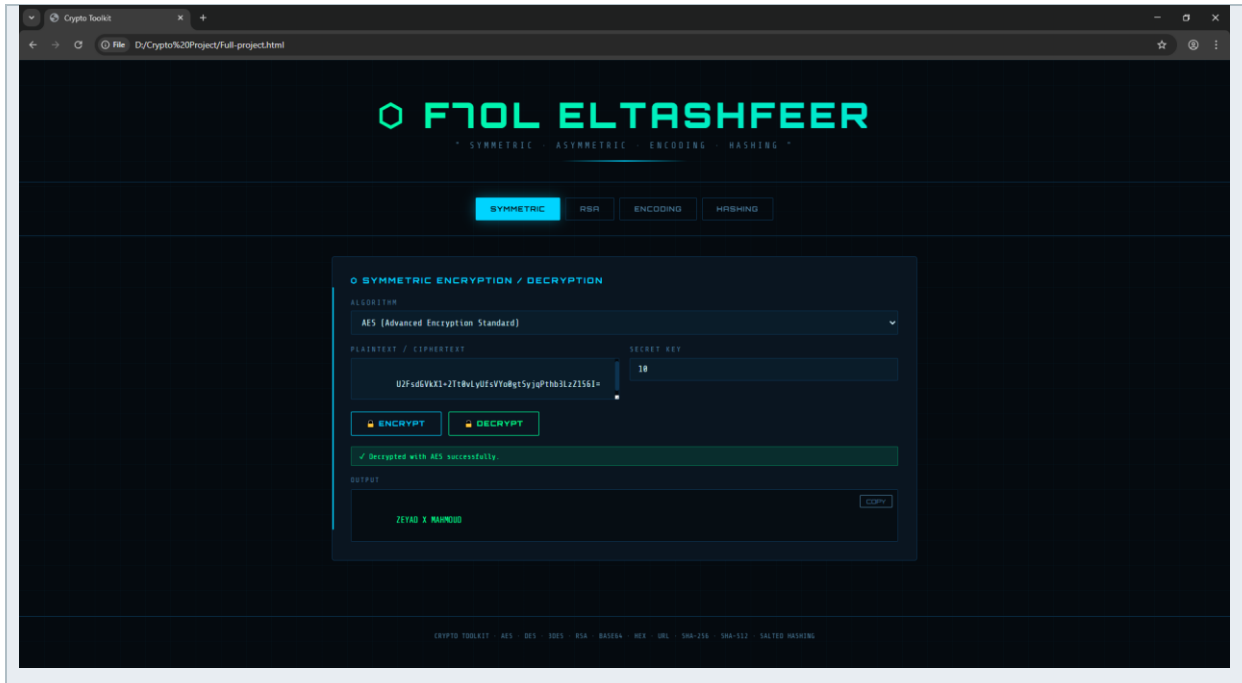


## RSA Encryption

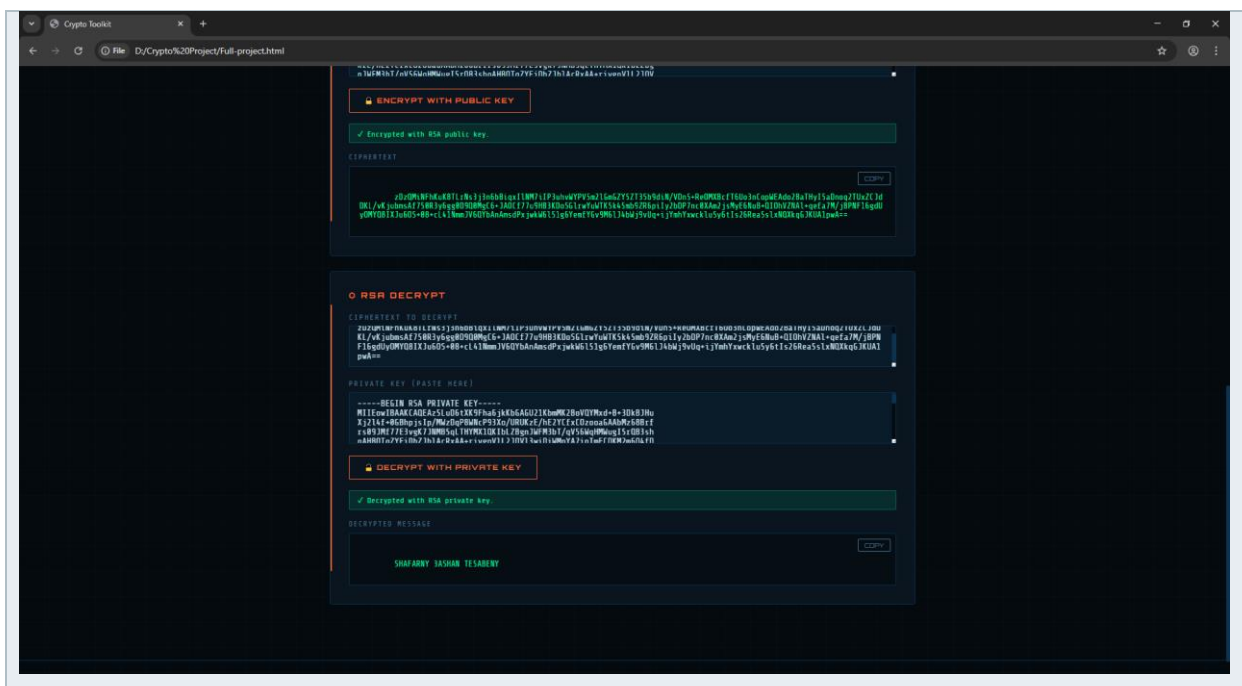


## 2.2 Decryption Screenshots

### AES Decryption

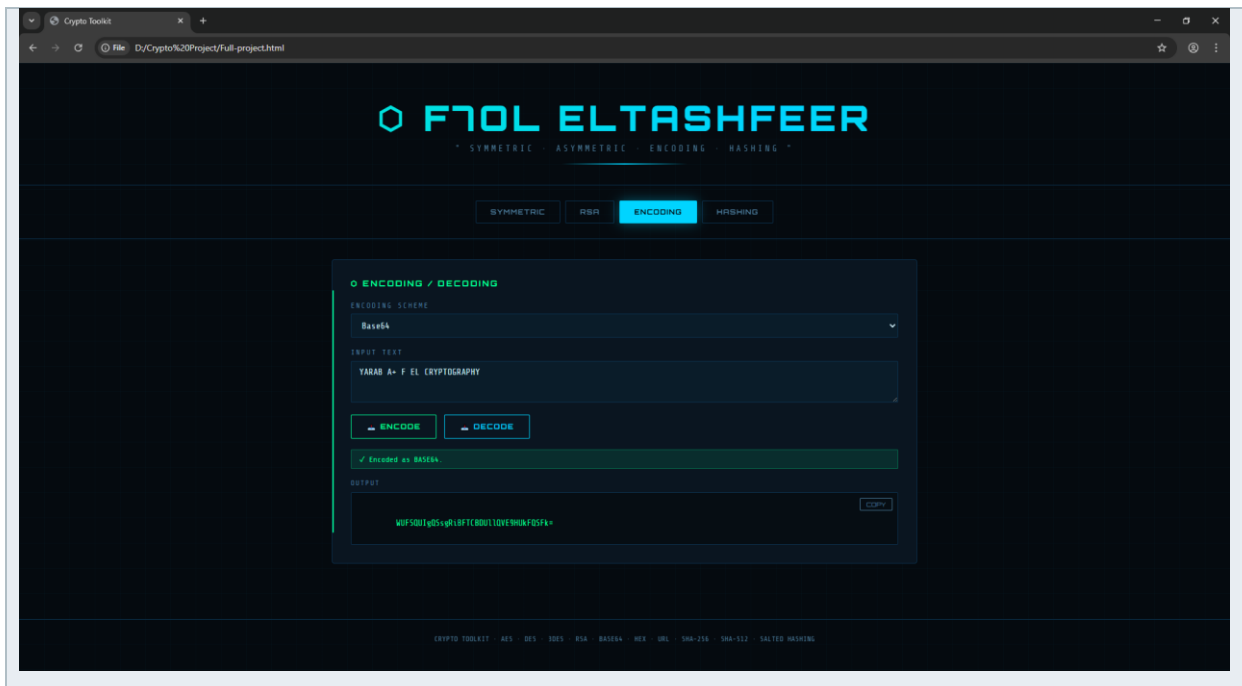


### RSA Decryption

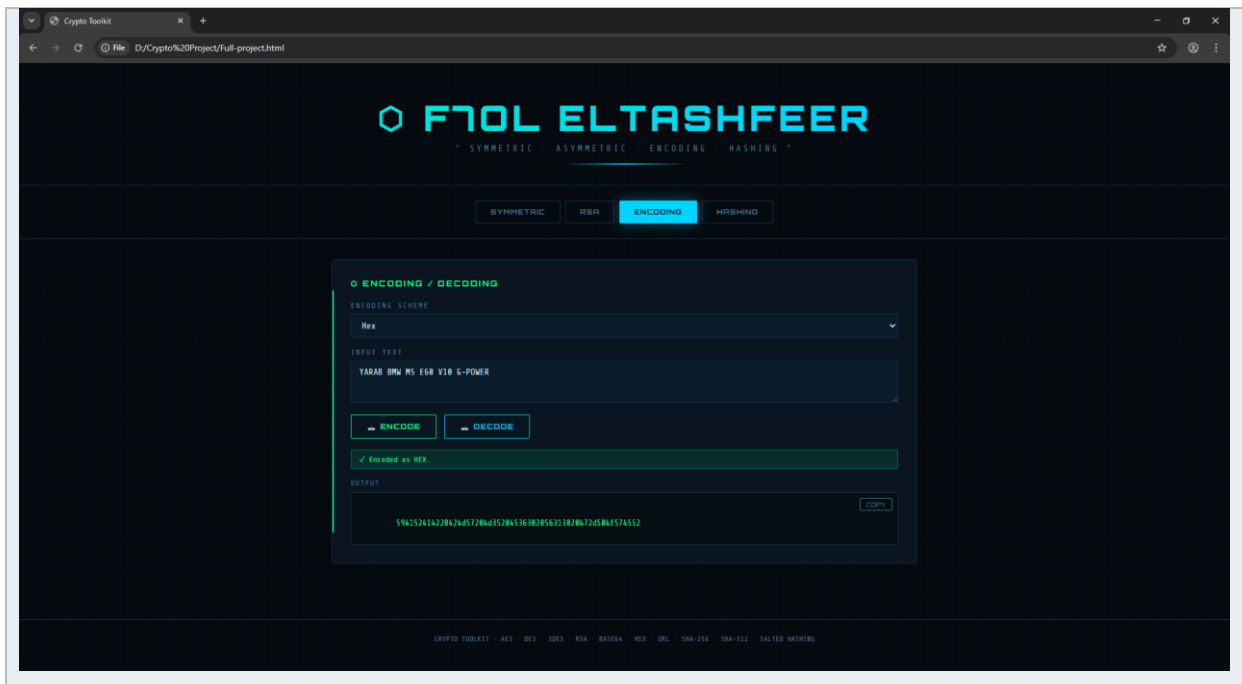


## 2.3 Encoding Screenshots

### Base64 Encoding

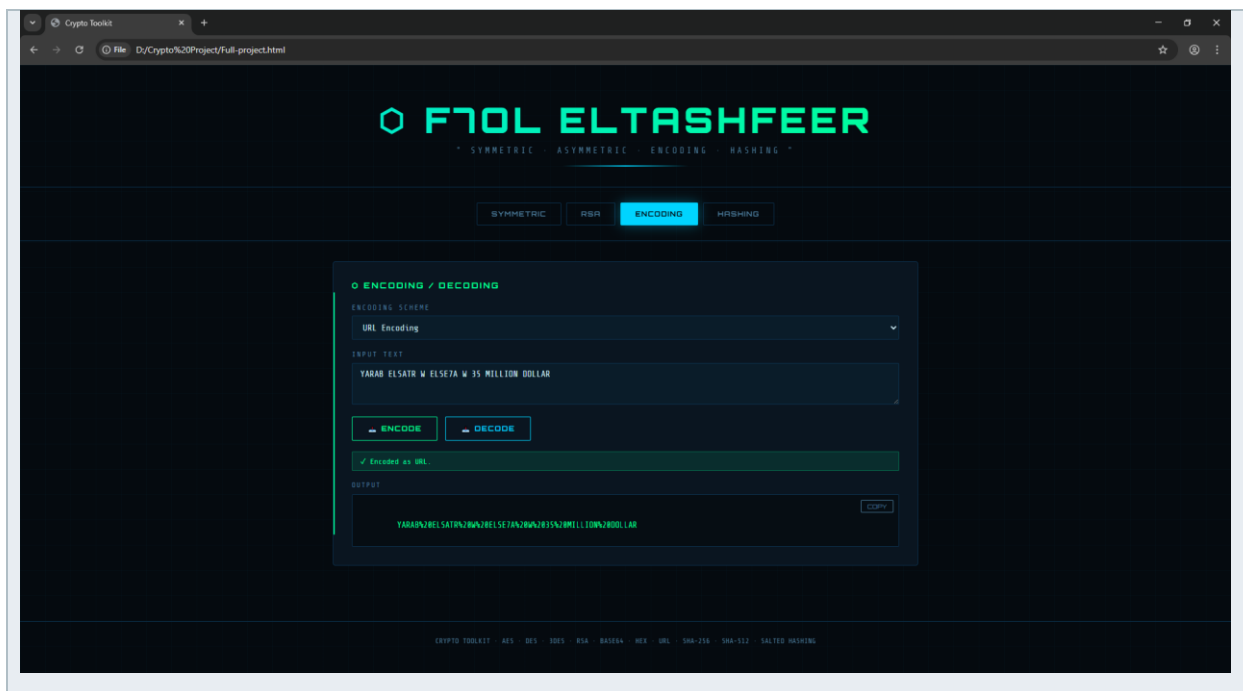


### Hex Encoding



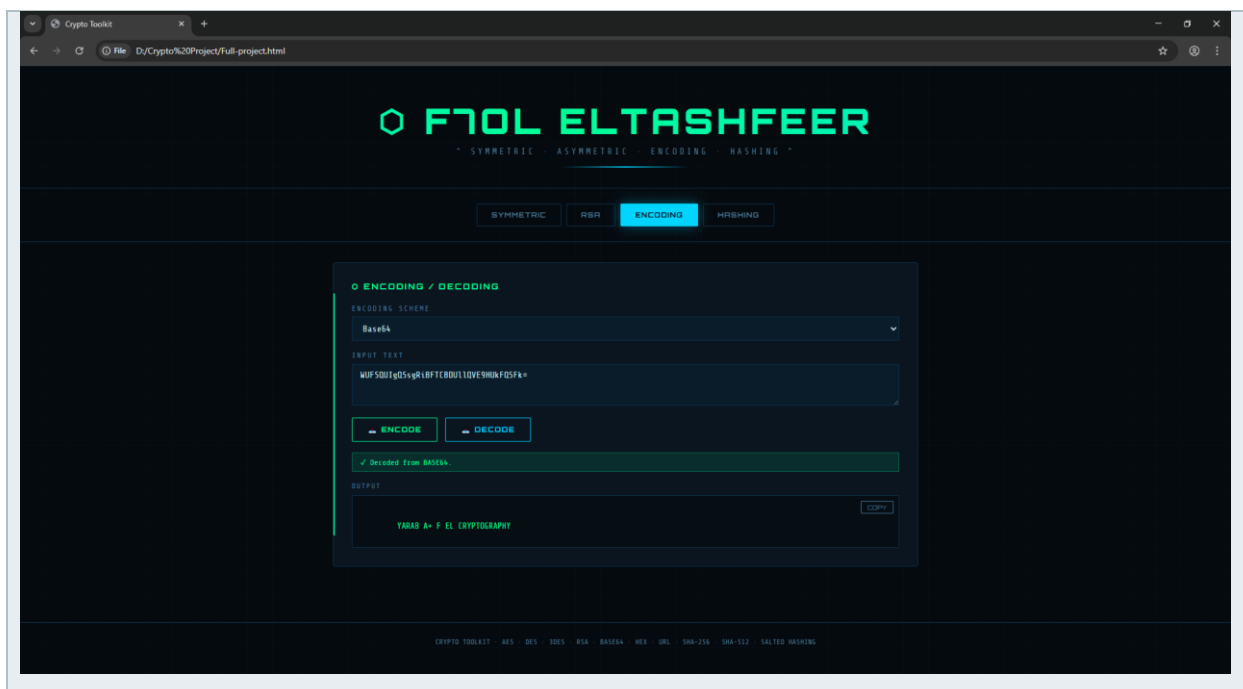


## URL Encoding

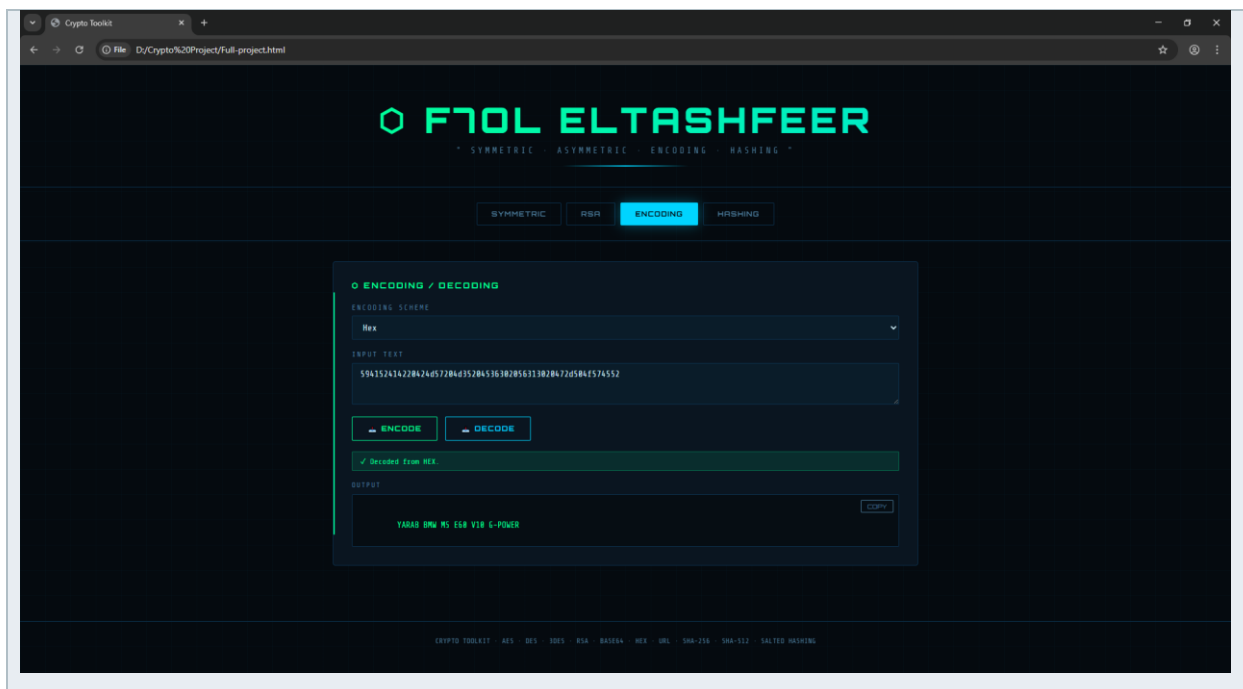


## 2.4 Decoding Screenshots

### Base64 Decoding

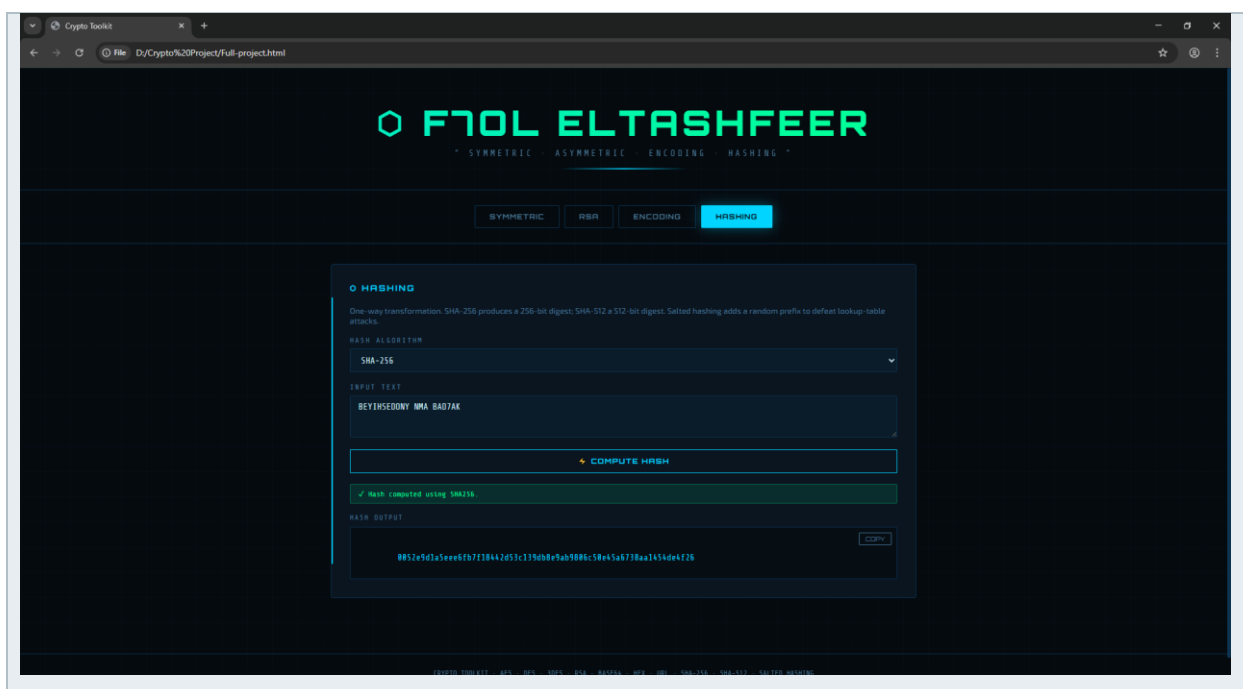


## Hex Decoding



## 2.5 Hashing Screenshots

### SHA-256 Hash



## SHA-512 Hash

The screenshot shows a web browser window with the URL `D:\Crypto%20Project\full-project.html`. The page features a dark theme with a green logo and navigation links: SYMMETRIC, RSA, ENCODING, and HASHING. The HASHING tab is active. Below the navigation bar, there is a section titled "HASHING" with a description: "One-way transformation, SHA-256 produces a 256-bit digest, SHA-512 a 512-bit digest. Salted hashing adds a random prefix to defeat lookup-table attacks." The "HASH ALGORITHM" dropdown is set to "SHA-512". The "INPUT TEXT" field contains "HELIF EL AMR YAMEN". A green button labeled "COMPUTE HASH" is visible. Below it, a green bar indicates "Hash computed using SHA512". The "HASH OUTPUT" section displays a long hexadecimal string: `2f71787b8e4e6315d858864e2b912f9688a5c3e4c580216dfe24e1182ca2e78a548f3a29b5ccfe2d994ef2ac137a816773b84a9352a2a5867a82e0deba2`. A "COPY" button is located next to the output.

## Salted Hashing

The screenshot shows the same web browser window, but the "HASHING" section is now configured for salted hashing. The "HASH ALGORITHM" dropdown is set to "Salted Hashing (SHA-256 + random salt)". The "INPUT TEXT" field contains "NAS JAYSIA W NAS EL MYAN BAYSIAN". The "SALT (AUTO-GENERATED OR CUSTOM)" field contains a long hexadecimal string: `ea5234430db0f23793a385a97c23745`. A "NEW SALT" button is visible next to the salt field. A green button labeled "COMPUTE HASH" is visible. Below it, a green bar indicates "Hash computed using salted". The "HASH OUTPUT" section displays the salt and the resulting hash:   
SALT: `ea5234430db0f23793a385a97c23745`  
HASH: `4db28085b0878ab0931a39c31d090da0ba10d774d31966121132984f5681c0`  
A "COPY" button is located next to the output.

## Section 3 – Testing Examples

---

Each subsection provides a concrete test case showing the input, selected operation, and output.

### 3.1 Symmetric Encryption / Decryption

#### AES – Encrypt

|           |                                                                                         |
|-----------|-----------------------------------------------------------------------------------------|
| Input     | Hello, Cryptography!                                                                    |
| Operation | AES Encrypt   Key: mysecretkey123                                                       |
| Output    | U2FsdGVkX1+... (Base64 ciphertext - varies per run due to random IV)                    |
| Note      | AES uses a random IV each run, so the ciphertext changes even for the same input & key. |
|           |                                                                                         |

#### AES – Decrypt

|           |                                                    |
|-----------|----------------------------------------------------|
| Input     | (paste ciphertext from above)                      |
| Operation | AES Decrypt   Key: mysecretkey123                  |
| Output    | Hello, Cryptography!                               |
| Note      | Correct key recovers the exact original plaintext. |
|           |                                                    |

#### DES – Encrypt

|           |                                    |
|-----------|------------------------------------|
| Input     | TestMessage                        |
| Operation | DES Encrypt   Key: 12345678        |
| Output    | U2FsdGVkX1+... (Base64 ciphertext) |
|           |                                    |

#### 3DES – Encrypt

|           |                                             |
|-----------|---------------------------------------------|
| Input     | SecureData                                  |
| Operation | 3DES Encrypt   Key: my24bytekey1234567890ab |
| Output    | U2FsdGVkX1+... (Base64 ciphertext)          |
|           |                                             |

## 3.2 RSA Asymmetric Encryption / Decryption

### RSA – Key Generation

|           |                                                                                                                                                                                    |
|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Input     | (none – click Generate Key Pair)                                                                                                                                                   |
| Operation | RSA 2048-bit Key Generation                                                                                                                                                        |
| Output    | Public Key: -----BEGIN PUBLIC KEY-----<br>\nMIIBIjAN...\n-----END PUBLIC KEY-----<br>Private Key: -----BEGIN RSA PRIVATE KEY-----<br>\nMIIEowIBA...\n-----END RSA PRIVATE KEY----- |
|           |                                                                                                                                                                                    |

### RSA – Encrypt

|           |                                                                             |
|-----------|-----------------------------------------------------------------------------|
| Input     | Secret Message                                                              |
| Operation | RSA Encrypt   using generated Public Key                                    |
| Output    | hX3p7Q... (long Base64 string ~344 chars for 2048-bit key)                  |
| Note      | RSA ciphertext is always the same size as the key (256 bytes for 2048-bit). |
|           |                                                                             |

### RSA – Decrypt

|           |                                           |
|-----------|-------------------------------------------|
| Input     | (RSA ciphertext from above)               |
| Operation | RSA Decrypt   using generated Private Key |
| Output    | Secret Message                            |
|           |                                           |

## 3.3 Encoding & Decoding

### Base64 – Encode

|           |                              |
|-----------|------------------------------|
| Input     | Cryptography is fun!         |
| Operation | Base64 Encode                |
| Output    | Q3J5cHRvZ3JhcGh5IGlzIGZ1biE= |
|           |                              |

### Base64 – Decode

|           |                              |
|-----------|------------------------------|
| Input     | Q3J5cHRvZ3JhcGh5IGlzIGZ1biE= |
| Operation | Base64 Decode                |
| Output    | Cryptography is fun!         |

|  |  |
|--|--|
|  |  |
|--|--|

### Hex – Encode

|           |                                                            |
|-----------|------------------------------------------------------------|
| Input     | Hello                                                      |
| Operation | Hex Encode                                                 |
| Output    | 48656c6c6f                                                 |
| Note      | H=48, e=65, l=6c, l=6c, o=6f (ASCII values in hexadecimal) |
|           |                                                            |

### Hex – Decode

|           |            |
|-----------|------------|
| Input     | 48656c6c6f |
| Operation | Hex Decode |
| Output    | Hello      |
|           |            |

### URL Encoding – Encode

|           |                                |
|-----------|--------------------------------|
| Input     | Hello World & welcome!         |
| Operation | URL Encode                     |
| Output    | Hello%20World%20%26%20welcome! |
| Note      | Space → %20, & → %26           |
|           |                                |

### URL Encoding – Decode

|           |                                |
|-----------|--------------------------------|
| Input     | Hello%20World%20%26%20welcome! |
| Operation | URL Decode                     |
| Output    | Hello World & welcome!         |
|           |                                |

## 3.4 Hashing

### SHA-256

|           |                                                                          |
|-----------|--------------------------------------------------------------------------|
| Input     | password123                                                              |
| Operation | SHA-256 Hash                                                             |
| Output    | ef92b778bafef771e89245b89ecbc08a44a4e166c06659911881f383d4473e94f        |
| Note      | 64 hexadecimal characters. Output is always the same for the same input. |

|  |  |
|--|--|
|  |  |
|--|--|

## SHA-512

|                  |                                                                                                                                  |
|------------------|----------------------------------------------------------------------------------------------------------------------------------|
| <b>Input</b>     | password123                                                                                                                      |
| <b>Operation</b> | SHA-512 Hash                                                                                                                     |
| <b>Output</b>    | b109f3bbbc244eb82441917ed06d618b9008dd09b3befd1b5e07394c706a8bb980b1d7785e5976ec049b46df5f1326af5a2ea6d103fd07c95385ffab0cacbc86 |
| <b>Note</b>      | 128 hexadecimal characters. Longer digest = higher security margin.                                                              |
|                  |                                                                                                                                  |

## Salted SHA-256

|                  |                                                                                               |
|------------------|-----------------------------------------------------------------------------------------------|
| <b>Input</b>     | mypassword                                                                                    |
| <b>Operation</b> | Salted Hashing (SHA-256 + random 16-byte salt)                                                |
| <b>Output</b>    | SALT: a3f9b2c107d4e581fa962b3c408d1e7f<br>HASH: 9d4e1e23c8... (64 chars)                      |
| <b>Note</b>      | Running the same operation twice gives a DIFFERENT hash because the salt is random each time. |
|                  |                                                                                               |